

Exactness of ApproxLP on Chain-Graph LCNs

Why ApproxLP is an *inner* bound of the strong extension, when the inner bound is nonetheless tight, and how it pairs with Credal VE to bracket the LCN truth

IBM Research

Abstract

ApproxLP (`lcn/inference/marginal/cn/approxlp.py`, class `ApproxLP`) is the coordinate-descent member of the credal-network family: it reformulates credal marginal inference as a multilinear program and solves it by iterated linearization — fixing all local distributions but one and picking that family’s best extreme point, sweeping to a local optimum (Antonucci, de Campos, Huber & Zaffalon, 2015). This note pins down three things about ApproxLP against the implementation. **(i) What it bounds.** Like every credal-network engine, ApproxLP works over the *strong extension* $\mathcal{D}_{SE}$ of the compiled network, not the LCN’s distribution set $\mathcal{D}_{LCN}$. Uniquely in the family, it is an *inner* approximation: because coordinate descent visits only *feasible* product joints and converges to a *local* optimum of a nonconvex multilinear program, the returned interval sits *inside* the strong-extension interval $[q_{SE}, \bar{q}_{SE}]$. It is therefore *not* a valid outer bound on $\mathcal{D}_{LCN}$; it can be too tight. **(ii) When the inner bound is tight.** When every per-variable linearization has its global optimum reachable by coordinate descent — which holds on the singly-connected, sentence-free instances where $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ and the multilinear objective is benign — ApproxLP’s inner interval *coincides* with the exact bound. We verify this on `d4_biting`: ApproxLP returns $P(A)=[0.40, 0.60]$, $P(B)=[0.04, 0.72]$, $P(C)=[0.08, 0.80]$, matching Credal VE and the exact oracle to four decimals — a sound, tight inner bound (`IntervalBP`, after its two message-passing bugs were fixed, now matches these too; `docs/ibp_exactness.tex`). The inner gap becomes strict only when coordinate descent stalls at a non-global local optimum. **(iii) Which tightening schemes apply.** `D1/D2/D3` are *build-stage* knobs that ApproxLP inherits for *free* (tighter vertices $\Rightarrow$ tighter LPs). `D4` (`coupling="cross-family"`) is implemented, as extra LP rows that keep every visited joint feasible — but it is *experimental and not a certified* bound: when the center initialization already violates a constraint, descent can stall at a degenerate point. `D5` is the separate exact engine `CredalJT`. The natural use of ApproxLP is as the *inner* half of a two-sided bracket: ApproxLP (inner) with Credal VE (outer) on the *same* vertices sandwiches the LCN truth.

Contents

1	Background: the ApproxLP algorithm	2
1.1	LCNs, the credal network, and the strong extension	2
1.2	Multilinear program and coordinate descent	2
2	What ApproxLP bounds: inner over the strong extension	3
3	When the inner bound is tight	3
3.1	Empirical confirmation on <code>d4_biting</code>	3
3.2	Topology by topology	4
4	Pairing with Credal VE: a two-sided bracket	4

5	The tightening schemes (D1–D5) for ApproxLP	5
6	Verdict	5

1 Background: the ApproxLP algorithm

1.1 LCNs, the credal network, and the strong extension

An LCN over binary atoms $\mathbf{V} = \{V_1, \dots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$, over propositional formulas. With the conditional independencies of the *Local Markov Condition* (LMC) of the primal graph, they define a set $\mathcal{D}_{\text{LCN}} \subseteq \Delta(2^{\mathbf{V}})$ of joints, and the marginal task for a query atom a is $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P(a)$, $\overline{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P(a)$.

The `cn/` pipeline compiles the chain-graph LCN into a *credal network*: a directed graph over the families $\text{ch}_v \mid \text{pa}_v$ with interval-valued local credal sets K_v . `CredalNetworkVertices.from_lcn` enumerates the extreme points of every K_v (`cnv.extreme_points`); all engines — Credal VE, CCTE, IBP, ApproxLP, Credal IJGP — consume that object. What they bracket is the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}, \quad (1)$$

and, because the chain-graph factorization reproduces the LMC exactly,

$$\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}, \quad (2)$$

with equality unless a *cross-family sentence* couples choices the free product decouples. Credal VE returns *exactly* $[\underline{q}_{\text{SE}}, \overline{q}_{\text{SE}}]$ (`docs/cve_exactness.tex`); ApproxLP approximates the *same* target *from the inside*.

1.2 Multilinear program and coordinate descent

Credal marginal inference over $\mathcal{D}_{\text{SE}}$ is a *multilinear program*: the objective $P(\text{query} \mid \text{evidence})$ is a ratio that is linear in each family's distribution P_v when the others are fixed, and the feasible region is the product of the polytopes K_v (`approxlp.py:41--49`). ApproxLP (`approxlp.py:306--380`, `_coordinate_descent`) solves it by iterated linearization:

1. **Initialize** every local P_v to the *center* (mean of its extreme points) (`approxlp.py:112--131, 318`).
2. **Sweep** (`approxlp.py:332--370`): for each family v and each parent configuration, hold all other families fixed; the objective is then linear in P_v , so its optimum over the polytope K_v is at a *vertex*. Enumerate the extreme points, evaluate the objective for each by an exact variable-elimination pass on the resulting precise Bayes net (`approxlp.py:133--234`), and *greedily commit* the best (`approxlp.py:359--370`).
3. **Converge** (`approxlp.py:375--378`): stop when a full sweep improves nothing, or after `n_iters` (default 50).

The run is repeated with `sense="min"` and `"max"` to get the lower and upper bound of each query. The public API is `run(evidence, n_iters=50, coupling="off", verbosity=1)` (`approxlp.py:386--389`).

2 What ApproxLP bounds: inner over the strong extension

Proposition 1 (ApproxLP is an inner bound). *With `coupling="off"`, ApproxLP returns per-atom intervals contained in the strong-extension interval: $[\underline{q}_{\text{ALP}}, \bar{q}_{\text{ALP}}] \subseteq [\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$. In general it is therefore not a valid outer bound on $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Every state ApproxLP evaluates is a *single feasible product joint*: at every point of the sweep each family sits at one extreme point of its K_v (or at the center, itself a convex combination of them), so the assembled joint is a point of $\mathcal{D}_{\text{SE}}$ ((1)). The objective is evaluated exactly on that precise Bayes net (`approxlp.py:133--234`). Hence every value ApproxLP ever records is $P(a)$ for some $P \in \mathcal{D}_{\text{SE}}$, and lies in $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$. The reported lower (resp. upper) bound is the smallest (largest) such value found along the descent trajectory, which is a min (resp. max) over a *subset* of $\mathcal{D}_{\text{SE}}$ and therefore no smaller (no larger) than the true $\mathcal{D}_{\text{SE}}$ extremum. This is the exact opposite of Credal VE, which enumerates *all* vertex combinations and so attains the global $\mathcal{D}_{\text{SE}}$ extremum: $\underline{q}_{\text{SE}} \leq \underline{q}_{\text{ALP}} \leq \bar{q}_{\text{ALP}} \leq \bar{q}_{\text{SE}}$. $\square$

The inner gap is strict exactly when coordinate descent *stalls*: the multilinear program is nonconvex, so a greedy sweep from the center can reach a local optimum that is not the global $\mathcal{D}_{\text{SE}}$ extremum (no single-family move improves, yet a coordinated multi-family jump would). The inner gap is *zero* when the reachable local optimum is global — the tightness question of Section 3.

3 When the inner bound is tight

By Proposition 1, ApproxLP equals the exact LCN bound when *both* (a) its inner interval coincides with $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$ (coordinate descent reaches the global $\mathcal{D}_{\text{SE}}$ extremum) *and* (b) $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ (singly connected, no cross-family sentence, as for Credal VE). Condition (b) is the same geometric condition analyzed in `docs/cve_exactness.tex`; condition (a) is an optimization-landscape property of the multilinear program.

Proposition 2 (Tightness on benign instances). *On a clean singly-connected, cross-family-sentence-free LCN whose per-family linearizations each expose their global optimum to a greedy sweep from the center, ApproxLP’s inner interval coincides with the exact LCN marginal bounds.*

Proof sketch. (b) gives $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ exactly as in `docs/cve_exactness.tex` (a chain-graph product satisfies every LMC; with no cross-family sentence the free product cannot leave $\mathcal{D}_{\text{LCN}}$). Under (a), the descent reaches the global $\mathcal{D}_{\text{SE}}$ extremum, so by Proposition 1’s chain of inequalities the inner interval equals $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}] = [\underline{q}_{\text{LCN}}, \bar{q}_{\text{LCN}}]$. $\square$

3.1 Empirical confirmation on d4_biting

`examples/d4_biting.lcn` (three atoms $A \rightarrow B, A \rightarrow C$; families $P(A), P(B \mid A), P(C \mid A)$; one cross-family sentence $P(B \wedge C) \leq 0.05$) is the family’s standard stress instance. The exact unconditional bounds are $P(A) = [0.40, 0.60]$, $P(B) = [0.04, 0.72]$, $P(C) = [0.08, 0.80]$ (four independent oracles agree; `docs/ccte_exactness.tex`). Measured, with `coupling="off"`:

$$\text{ApproxLP: } P(A)=[0.40, 0.60], \quad P(B)=[0.04, 0.72], \quad P(C)=[0.08, 0.80],$$

matching the exact bounds to four decimals. The engines on the *same* vertices:

Engine on d4.biting	$P(A)$	$P(B)$	$P(C)$
Exact = Credal VE = oracle	[0.40, 0.60]	[0.04, 0.72]	[0.08, 0.80]
ApproxLP (coupling="off")	[0.40, 0.60]	[0.04, 0.72]	[0.08, 0.80]
IntervalBP (interval, fixed)	[0.40, 0.60]	[0.04, 0.72]	[0.08, 0.80]
CCTE (coupling="off")	[0.40, 0.60]	[0.05, 0.65]	[0.08, 0.80]

On this instance ApproxLP’s inner interval reaches the exact bound, and **IntervalBP** matches it too now that its two message-passing bugs are fixed (before the fix it returned the inner $[0.5, 0.5]$ / $[0.05, 0.65]$ / $[0.10, 0.75]$; `docs/ibp_exactness.tex`). CCTE remains inner on $P(B)$ (`docs/ccte_exactness.tex`). The reason ApproxLP succeeds here is that, for these unconditional marginals, the objective’s global $\mathcal{D}_{SE}$ extremum *is* a coordinate-descent fixed point reachable from the center — condition (a) holds. On harder (nonconvex, more coupled) instances condition (a) can fail and the inner gap becomes strict; then ApproxLP under-reports the width and must be read as an inner bound only.

3.2 Topology by topology

Markov chain, tree, polytree. Singly connected, no cross-family sentence, so $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$; the per-family linearizations are simple and coordinate descent reaches the global extremum, so ApproxLP is tight (equal to **ExactInference** / Credal VE) on `examples/chain.lcn`, `tree.lcn`, `polytree.lcn`. Verified on a two-atom chain $x_0 \rightarrow x_1$: ApproxLP returns $P(x_0)=[0.300, 0.500]$, $P(x_1)=[0.400, 0.680]$, matching Credal VE exactly (the fixed **IntervalBP** now agrees on this instance too; `docs/ibp_exactness.tex`).

Chain graph with compound nodes (alarm). Still no cross-family sentence, so $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$; ApproxLP is tight provided descent does not stall on the larger compound-node linearizations. This is the regime where pairing with Credal VE (Section 4) is most valuable as a stall detector.

Loopy DAG / cross-family sentence. $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$ (or descent stalls). ApproxLP remains an *inner* bound of $\mathcal{D}_{SE}$; it never over-covers, so it cannot certify an outer bound on $\mathcal{D}_{LCN}$. Use it as the inner half of a bracket, and use D5 (**CredalJT**) for a guaranteed exact bound.

4 Pairing with Credal VE: a two-sided bracket

Because ApproxLP is *inner* over $\mathcal{D}_{SE}$ and Credal VE is *exact* over $\mathcal{D}_{SE}$, running both on the *same* enumerated vertices yields

$$\underbrace{[q_{ALP}, \bar{q}_{ALP}]}_{\text{inner}} \subseteq \underbrace{[q_{SE}, \bar{q}_{SE}]}_{\text{Credal VE, exact over } \mathcal{D}_{SE}} \supseteq \underbrace{[q_{LCN}, \bar{q}_{LCN}]}_{\text{truth, when } \mathcal{D}_{SE}=\mathcal{D}_{LCN}} .$$

When the two intervals *coincide*, coordinate descent reached the global $\mathcal{D}_{SE}$ extremum and the common value is certified as the exact $\mathcal{D}_{SE}$ bound; when they *differ*, the gap between them is precisely the coordinate-descent stall, flagging that ApproxLP under-reported the width. This is the recommended diagnostic use of ApproxLP (`docs/tighter_approximation.tex`): the inner ApproxLP and outer Credal VE bracket the LCN truth from both sides on the tightened vertices.

5 The tightening schemes (D1–D5) for ApproxLP

The companion note `docs/tighter_approximation.tex` identifies **Gap A** (local-set widening) and **Gap B** (strong-extension widening) and five schemes.

Class 1 — input-level (D1, D2, D3): free build-stage knobs. These change only what `CredalNetworkVertices.from_lcn` produces; ApproxLP consumes tighter vertices without any code change, and its per-variable LP already optimizes over the resulting (smaller) vertex polytope, so its inner interval tightens toward $\mathcal{D}_{\text{LCN}}$ from the inside. **D1** (`method="linear-tight"`) enriches each family LP with in-scope sentences and scope-restricted LMC equalities; **D2** (`merge_budget=k`) merges adjacent families; **D3** (`solver="scip"`) certifies each family’s local set globally.

Class 2 — propagation-level coupling (D4, D5).

- **D4** (`coupling="cross-family"` on `ApproxLP.run`): implemented (`approxlp.py:265--304, 346--350`) by rejecting any vertex pick whose fully reconstructed joint violates a cross-family constraint — i.e. adding D4’s coupling rows to the per-variable LP, which keeps the inner-bound guarantee while shrinking the polytope (`docs/tighter_approximation.tex`). **But it is not a certified bound:** the `run` docstring warns that ApproxLP “is an inner, greedy coordinate-descent method and is the loosest fit for D4”; when the center initialization already violates a constraint, descent can find no feasible improving move and *stall at a degenerate point* (e.g. reporting $[0.5, 0.5]$). For a trustworthy coupled bracket, use coupled Credal VE (outer) instead.
- **D5** (`CredalJT`): a separate junction-tree NLP engine, not reachable from ApproxLP; the reliable exact route for cross-family coupling and conditional queries.

Scheme	Where it lives for ApproxLP	Gap closed	Character	Cost
D1 <code>method="linear-tight"</code>	build (<code>from_lcn</code>)	A (partial)	free, inner tightens	≈ 0
D2 <code>merge_budget=k</code>	build (<code>from_lcn</code>)	A (+ B)	free	2^k
D3 <code>solver="scip"</code>	build (<code>from_lcn</code>)	A (exact)	free	$2^n/\text{solve}$
D4 <code>coupling="cross-family"</code>	inside <code>run</code> (LP rows)	B	experimental, not certified	med
D5 <code>CredalJT</code>	separate engine	A + B	exact, separate	treewidth

6 Verdict

Proposition 3 (Summary). *Let $\mathcal{L}$ be a chain-graph LCN and run `ApproxLP`.*

1. (Always) With `coupling="off"`, `ApproxLP` returns an inner approximation of $[q_{\text{SE}}, \bar{q}_{\text{SE}}]$ — not a valid outer bound on $\mathcal{D}_{\text{LCN}}$; it can be too tight (Proposition 1).
2. (Tight) When $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ (singly connected, no cross-family sentence) and coordinate descent reaches the global $\mathcal{D}_{\text{SE}}$ extremum, the inner interval coincides with the exact LCN bounds (Proposition 2); verified on `d4_biting`, where `ApproxLP` matches the exact $P(B) = [0.04, 0.72]$ (as does the fixed `IntervalBP`; `CCTE` remains inner there).
3. (Strict gap) The inner bound is strict exactly when the nonconvex multilinear program makes descent stall at a non-global local optimum; then `ApproxLP` under-reports the width.

4. (Bracket) *Pairing ApproxLP (inner) with Credal VE (exact over $\mathcal{D}_{SE}$) on the same vertices brackets the truth and detects stalls (Section 4).*
5. (Tightening) *D1/D2/D3 tighten ApproxLP for free at build time; D4 is implemented but experimental and uncertified; D5 (CredalJT) is the separate exact engine.*

Placed against its siblings: Credal VE and CCTE ($\epsilon = 0$) are *exact* over $\mathcal{D}_{SE}$; IntervalBP is *outer* (interval) / *inner* (variational) — exact on trees now that its message bugs are fixed; ApproxLP is the family’s *inner* member — and, on the instances tested here, a *sound and tight* one that is the natural inner half of a two-sided bracket. The three companion notes cover the neighbors: `docs/cve_exactness.tex` (the exact-over- $\mathcal{D}_{SE}$ per-target engine), `docs/ibp_exactness.tex` (the message-passing engine), and `docs/tighter_approximation.tex` (the D1–D5 taxonomy and the D5 exactness theorem).